RESEARCH

Reinforcement Learning for Single-Stock Portfolio Optimization in Equity Markets

Cihang Huang · Zhucui Jing

Received: 30 June 2025 / Revised: 20 November 2025 / Accepted: 30 November 2025
© The Author(s) 2025

Abstract
In today's volatile financial markets, flexible asset allocation and portfolio management are crucial for maximizing returns and lowering risk. Real-time portfolio allocation optimization is achieved by the Reinforcement Learning-based Adaptive Portfolio Management (RL-APM) system using the Deep Deterministic Policy Gradient (DDPG) approach. A state vector representing volatility, recent returns, and current portfolio weights across a range of equity assets is produced by RL-APM, which also analyzes daily market data, including Open, High, Low, Close, and Volume (OHLCV) values, and calculates technical indicators such as SMA, RSI, MACD, and Bollinger Bands. This state is used by the DDPG agent to create continuous portfolio weights that dynamically adapt to market fluctuations. In contrast to traditional DDPG-based techniques, the method introduces a novel multi-objective reward function that simultaneously maximizes volatility, Sharpe ratio, cumulative return, and transaction costs, improving stability and realism. RL-APM outperformed benchmark models like Mean-Variance Optimization (MVO) using daily S&P 500 index data from 2015 to 2024. Average returns increased by 12.4% and the Sharpe ratio by 15.7%. These results demonstrate that the system may scale efficiently for real-world financial scenarios, such as hedge funds, automated trading, and robo-advisory platforms, while also creating flexible allocation plans and maintaining robust risk management.

Keywords Reinforcement learning · Dynamic asset allocation · Deep deterministic policy gradient · Sharpe ratio optimization · Adaptive investment strategy · Machine learning in finance

1 Introduction

The present article examines the drawbacks of traditional portfolio strategies, such Mean-Variance Optimization (MVO) [1], which are based on assumptions and cannot adapt to modern, complex markets. Further contributing to the limitations of existing reinforcement learning (RL) models is the fact that they often only deal with simple or single-asset scenarios, leaving out real-world elements such as transaction costs and liquidity risks [2]. In order to solve these challenges, the proposed Reinforcement Learning–based Adaptive Portfolio Management (RL-APM) architecture continuously and intelligently adjusts multi-asset portfolios in real time using the Deep Deterministic Policy Gradient (DDPG) technique [3]. Returns, volatility, Sharpe ratio, and transaction costs are

International Journal of Computational Intelligence Systems (2026) 19:37
https://doi.org/10.1007/s44196-025-01105-x

Published online: 30 December 2025

all key variables that RL-APM incorporates into its reward function, providing a dynamic, scalable, and data-driven solution for dependable portfolio management in complex financial scenarios [4].

The initiative aims to create an adaptive portfolio management system that improves financial market asset al.location in real time. A reinforcement learning approach that adjusts portfolio weights across asset classes to maximize risk-adjusted returns while minimizing volatility and drawdown risk is needed [5]. Market indicators, portfolio characteristics, and economic variables make up the state space of the Markov Decision Process (MDP). In contrast, the action space represents weight changes, and the reward function represents the return-risk trade-off [6]. Finding a good learning algorithm that works with high-dimensional state and action spaces and changing financial markets is difficult.

Despite increased interest in applying machine learning to financial problems, real reinforcement learning solutions for dynamic portfolio management are lacking [7]. Most research has concentrated on simplified market environments or theoretical frameworks that don't handle real-world trading conditions [8]. Traditional mean-variance optimization usually assumes distributed returns, constant asset correlation, and correctly predicted return and covariance estimations [9]. These assumptions are often violated in real markets, leading to suboptimal portfolio performance and increased risk exposure. Recent studies have explored various machine learning approaches for portfolio optimization, but most treat portfolio optimization as a static problem, failing to capture the dynamic nature of financial markets [10]. Many existing reinforcement learning applications in finance focus on single-asset trading strategies rather than multi-asset portfolio management, which limits their practical utility for institutional investors and portfolio managers [11].

The extreme volatility of financial markets has highlighted the shortcomings of conventional techniques like Mean-Variance Optimization, which are predicated on unchangeable assumptions and are unable to adjust to evolving circumstances. The fact that many of the current reinforcement learning (RL) models only handle straightforward or single-asset scenarios and disregard real-world variables like transaction costs and liquidity is another disadvantage. This research introduces the Reinforcement Learning–based Adaptive Portfolio Management (RL-APM) system, which employs the DDPG algorithm for real-time, multi-asset portfolio adjustment in order to overcome these drawbacks. RL-APM is a dynamic, scalable, and risk-aware method that enhances portfolio performance in intricate financial circumstances by including returns, volatility, Sharpe ratio, and transaction costs into its reward function.

Multiple technical and computational hurdles hinder the development of reinforcement learning-based portfolio management solutions. Traditional portfolio optimization approaches are sensitive and unstable due to unclear parameters and static financial markets [12]. Dimensionality raises computational requirements for optimization, which they struggle with. Effective policies can be learned through environmental interaction using reinforcement learning [13]. Reinforcement learning can capture the interconnected decisions of portfolio managers over time [14]. Machine learning algorithms identify complex patterns and relationships in massive financial datasets, enhancing portfolio performance and risk management. Reinforcement learning models complicated, non-linear interactions among market variables, as well as portfolio performance [15].

This research concentrates on resolving the problems with current RL-based portfolio optimization models in order to bolster the work's originality. The majority of earlier DDPG and DRL approaches focus on simplified single-asset scenarios while ignoring the dynamics of multi-asset correlation, transaction costs, and liquidity limitations. A multi-objective reward function that concurrently maximizes return, volatility, Sharpe ratio, and transaction costs in a continuous action space is incorporated into the proposed Reinforcement Learning–based Adaptive Portfolio Management (RL-APM) system, in contrast to previous approaches. The RL agent may make risk-aware, in-the-moment portfolio modifications under dynamic market situations thanks to this structure. Thus, the suggested methodology provides a scalable, data-driven approach to adaptive portfolio management by bridging the gap between theoretical reinforcement learning models and actual financial trading settings.

An configurable incentive function during DRL is a unique feature of the system. This process weighs transaction costs, volatility, and risk-adjusted returns. RL-based portfolio optimization is not new, but the method offers a novel perspective. The RL-APM system was designed to handle multi-asset portfolios and make real-time asset

allocation changes. Since financial markets are always changing, this paradigm is more adaptable and durable. However, many RL-based systems simplify situations or specialize on a specific item. This method is unique since it accounts for real-world complications and frictions. It is ideal for institutional portfolio management. Reinforcement Learning-based Adaptive Portfolio Management (RL-APM) uses deep reinforcement learning adjusts portfolio strategies in real time. The main objectives are:

- To develop an RL-APM framework using the DDPG algorithm that adapts to changing market conditions and optimizes long-term investment returns.
- To compare the performance of the RL-based strategy against the traditional Mean-Variance Optimization model using key financial metrics such as cumulative return, Volatility, and Sharpe ratio.
- To demonstrate the practical applicability of the RL-APM framework in real-world financial environments, including stock trading, robo-advisory systems, and hedge fund asset allocation.

RL-APM uses DDPG-based reinforcement learning to optimize multi-asset portfolios in real time so that they can respond to changes in the financial markets. RL-APM improves risk-return performance by changing with the market while keeping some things the same. It takes into account both transaction costs and volatility when calculating rewards. Its data-driven and flexible approach to managing portfolios in the real world makes it more scalable and resilient. Using the Yahoo Finance dataset, demonstrate that the proposed Reinforcement Learning–based Adaptive Portfolio Management (RL-APM) framework, which is based on the DDPG algorithm, performs 15.7% better in the Sharpe ratio and 12.4% better in average return than the traditional Mean-Variance Optimization (MVO) benchmark. This illustrates how flexible, scalable, and useful the technology is for managing dynamic portfolios in the real world. Static and rule-based portfolio management and their limitations in uncertain financial markets are covered in Sect. 2. Section 3 describes the DDPG-based RL-APM architecture using real-time market data. Section 4 provides the historical asset dataset experimental setup, variance optimization model comparison, and performance results. Section 5 reviews the findings and suggests future directions, such as integrating complex reinforcement learning models and dynamic financial settings.

2 Related Work

Olanrewaju et al. [16] observed that machine learning may improve dynamic asset al.location in volatile U.S. markets. Deep reinforcement learning (DRL), neural networks, and random forest ensembles are compared to Markowitz mean-variance and BlackLitterman models. Maximize risk-adjusted returns while minimizing drawdowns. Data shows that ML-based portfolios outperform static benchmarks in key criteria. The DRL agent reduces equity exposure before volatility surges, thereby improving Sharpe ratios and mitigating drawdowns. These findings suggest AI-driven techniques can adapt to tumultuous times and outperform. The paper concludes with implications for investors, validation needs, and future research on explainable AI and financial theory.

Sutiene et al. [17] addressed the continual issue of establishing an investing portfolio that balances risk and reward using stocks, bonds, and cash. The portfolio management process involves strategy, execution, and feedback. AI has transformed asset al.location, and this article evaluates current proposals to transform portfolio management. Artificial intelligence in finance raises transparency, fairness, and explainability issues, and this work examines post-hoc justifications for asset al.location. The report also covers European investment industry legal changes and how explainable artificial intelligence could increase investment transparency.

Qiu et al. [18] proposed an intelligent Quantum Finance-based portfolio investment system (QFPIS), combining Deep Reinforcement Learning (DRL) with Quantum Finance Theory (QFT). The system comprises two major agents: a trading agent that utilizes the Deep Deterministic Policy Gradient algorithm for investment weighting and an intelligent agent that employs the Policy Gradient algorithm for risk control. This two-agent system design provides stable and realistic fund allocation for different financial products in the portfolio. Experiment results show robustness, flexibility, and profitability compared to other RL trading systems.

Owoade et al. [19] used predictive analytics and scalable data modeling in financial portfolio management. It precisely predicts asset trends, market anomalies, and portfolio risk using machine learning algorithms, big data architectures, and real-time data processing. Financial prediction is tested using timeseries forecasting, neural networks, and ensemble approaches. Apache Spark and cloud-based data lakes are covered. It found that predictive analytics and comprehensive data models can provide fund managers and institutional investors with real-time insights to enhance decision-making. Financial institution scalable model best practices are also included in the study.

Addy et al. [20] transformed financial market trading and risk management, discussing the evolution of algorithmic trading using machine learning (ML) algorithms, such as neural networks, decision trees, and ensemble approaches. For robust trading systems, address issues such as overfitting, data bias, and model interpretability. Market fairness and stability are ethical and regulatory considerations. ML is tested for market pattern identification, portfolio risk measurement, and risk management asset al.location. The assessment found that ML has altered financial markets, but responsible and sustainable integration needs addressing basic issues and ethics. The examination illuminates financial market ML and establishes the framework for algorithmic trading and risk management research in Table 1.

Huang et al. (2024) [29] used the Nonlinear-Activated Beetle Antennae Search and Egret Swarm Optimization Algorithm to address the Mean-Variance Portfolio Selection issue with Transaction Costs and Cardinality Constraints (MVPS-TCCC). Their strategy optimises portfolio designs to maximise the Sharpe Ratio while fulfilling strict limitations, resulting in performance closer to the portfolio's efficient frontier. We can improve our financial work by leveraging the asset al.location and risk-return trade-off knowledge gained from this study.

Harry Markowitz came up with portfolio optimization over 70 years ago. Portfolio return standard deviation is a way to trade risk and expected return. When it came out, MPT changed the way people designed quantitative portfolios. Markowitz's strategy [30]is utilized because it is simple and mathematically sound, even if it can lead to wrong estimates of return statistics. To make the theory more useful, it now covers transaction costs, constraints on leverage, and tax issues. MPT has trouble with unclear projections of return data. Machine learning, robust optimization, and Bayesian estimation are some of the new tools that can deal with parameter uncertainty. Convex optimization makes sure that solutions are quick and reliable. This strategy can now handle and respond to changes in the market, which solves many problems with managing a portfolio.

The MVO model assumes returns follow a normal distribution, asset correlations are constant, and historical data may predict future returns. Unfortunately, MVO fares poorly in the real world because financial markets are non-linear, unpredictable, and frequently change. The RL-APM framework uses DRL to adjust portfolio strategies dynamically and learns from constant market feedback, unlike the MVO approach. RL-APM can optimize portfolio allocations in real time, independent of market volatility, unlike older models that cannot keep up with shifting financial landscapes. RL-APM considers real-time data, transaction costs, and liquidity constraints. However, many RL models in the literature adopt single asset or simpler market strategies. RL-APM thinks all of these criteria. By changing its rules through market interactions, RL-APM optimizes the portfolio for risk and return. RL-APM is more versatile than static models like MVO.

The proposed RL-APM approach fixes some of the issues in RL and dynamic portfolio optimization studies. Early research employing Deep RL and Q-learning showed asset allocation success; however, these studies used simplified market conditions, single-asset strategies, or didn't account for real-time flexibility. RL-APM uses DDPG, a more robust and scalable continuous action space algorithm, to implement a real-time, continuous portfolio adjustment mechanism for multi-asset portfolios in unexpected markets. With transaction costs and volatility in the reward function, RL-APM is more flexible and data-driven than static models like MVO, which cannot adjust to market movements. With its revolutionary portfolio risk-return optimization strategy, RL-APM addresses multi-asset management and real-world trading issues in machine learning-based financial decision-making.

Table 1 Summary of recent research on RL-Based dynamic asset allocation and portfolio management

Authors	Objective	Methods	Findings	Advantages	Disadvantages	Research gap
Tursunalieva et al. [21]	Review interpretation techniques in ML and their financial applications	Systematic literature review on explainable ML techniques	Clarifies interpretability strategies relevant to finance models	Enhances model transparency and trust	Lacks focus on real-time portfolio adjustments	Limited discussion on RL-specific interpretability for portfolio strategies
Choi and Kim [22]	Optimize multi-asset portfolio using expert-infused DRL	Expert-infused DRL with market diversity constraints	DRL model outperforms traditional and baseline RL models	Incorporates human knowledge to stabilize learning	Complexity in expert modeling and generalization	Needs scalability testing in high-frequency trading
Cheng and Sun [23]	Manage multi-asset portfolios with multi-agent DRL	Multi-agent DRL framework for adaptive trading	Achieves robust performance in volatile markets	Decentralized agents improve diversification and reactivity	Coordination between agents adds system overhead	Lacks empirical benchmarking across diverse financial environments
Yoo and Gu [24]	Improve safety in RL-based dynamic asset allocation	AARL (Adaptive Action Reinforcement Learning) with safety-aware weight adjustments	Demonstrates risk-sensitive portfolio growth	Ensures safety and performance balance	May limit returns due to conservative adjustments	Limited real-world financial dataset validation
Espiga-Fernandez et al. [25]	Develop RL simulation environment for portfolio optimization	gymfolio – Python-based RL environment for financial training	Provides open-source, customizable portfolio simulation	Supports flexible testing and benchmarking	Requires programming proficiency and tuning	Absence of built-in real-world market scenarios or noise handling
Yan, K et al. [26]	To analyze volatility-based quantitative investment strategies for American financial stocks using machine learning.	Machine learning techniques for volatility forecasting and strategy optimization.	Predicts volatility trends and optimizes stock portfolios, outperforming traditional models in volatility prediction and risk management.	Provides data-driven insights that improve portfolio performance in volatile markets.	Limited to American financial stocks; performance under varying market conditions has not been thoroughly tested.	The approach needs to be extended to other asset classes and tested across multiple market regimes.
Bian et al. [27]	To introduce NGDRL, a dynamic news graph-based deep reinforcement learning (DRL) framework for portfolio optimization.	Utilizes a news graph-based DRL framework to incorporate news sentiment into portfolio decision-making.	NGDRL outperforms traditional models in portfolio allocation by using news sentiment as a dynamic feature.	Incorporates real-time news sentiment and enhances adaptability to news-driven market movements.	Complex implementation requires significant computational resources for news processing.	The framework could be expanded to incorporate a larger variety of market signals and real-world data streams.
Acero et al. [28]	To develop a hybrid model integrating deep reinforcement learning with mean-variance strategies for responsible portfolio optimization.	Combines DRL with mean-variance optimization to create a hybrid portfolio management strategy.	Balances risk and reward, providing responsible investment strategies that adapt to changing market conditions.	Offers a balanced risk-return trade-off, with an ethical and responsible portfolio strategy.	The model's complexity increases with market size, and it may face challenges with data sparsity in low-volume markets.	Need for further improvement in model scalability and application to multi-asset portfolios.

3 Reinforcement Learning-based Adaptive Portfolio Management

The intelligent investment system RL-APM manages portfolios and optimizes strategies using artificial intelligence. 24/7, it monitors market conditions, analyzes new data, makes real-time adjustments, and learns from every trade. The system adapts to market changes, crashes, and booms, and learns from mistakes. It utilizes current prices, volatility metrics, technical indicators, and historical judgments. The algorithm analyzes all market data, predicts outcomes, chooses portfolio weights to maximize projected returns, and ensures all investments total 100%. Cumulative returns, Sharpe ratios, maximum drawdown, and information ratios are measures of success.

RL-APM has real-world applications in individual investors, financial institutions, and various market environments, including bull, bear, volatile, and stable markets. Implementation considerations include high-performance computing resources, real-time data feeds, robust software infrastructure, backup systems, and regulatory compliance.

A systematic, data-driven approach to reinforcement learning and financial portfolio management is illustrated in the RL-APM framework's architectural Fig. 1. Interconnected modules process market data, make smart decisions, and enhance performance.

Data layer A multi-source data input layer includes the Yahoo Finance API for real-time market data, historical price databases for backtesting, live price feeds for current market conditions, market indices for benchmark comparisons, and comprehensive asset price repositories. This diversified data architecture assures market representation and allows the framework to adapt to different asset classes and market conditions.

Feature processing pipeline Advanced feature engineering computes technical indicators, price normalizations, and statistical metrics from raw market data. Current portfolio weights, market indicators, and risk measurements are combined into a mathematical framework by the State Construction module to create a structured state vector representation for reinforcement learning algorithms.

To help the reinforcement learning model use the data better, the RL-APM framework's Feature Engineering Pipeline cleans up financial data. First, get historical price series and real-time market data from Yahoo Finance. As part of data pre-treatment, the pipeline cleans, normalizes, and filters outliers. The data will be stable and consistent. Technical indications are extracted during feature engineering. These indicators include Bollinger Bands, MACD, SMA, and RSI. These indicators help analyse market conditions, price momentum, and volatility. Adding the processed features to a state vector that represents the market environment and portfolio performance completes the decision-making process. The reinforcement learning model incorporates this vector.

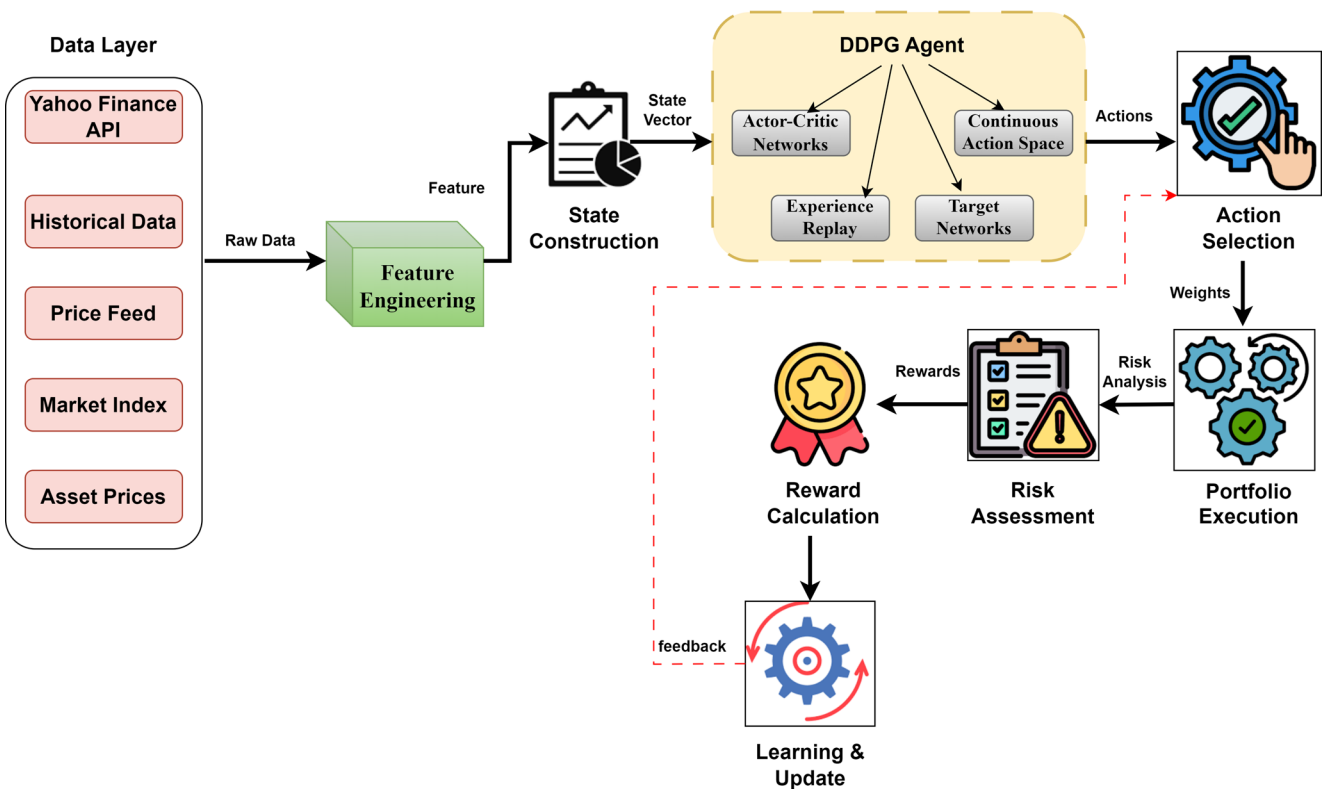

Fig. 1 Reinforcement learning-based adaptive portfolio management framework with DDPG agent

DDPG agent architecture Actor-Critic networks optimize policy and are its main intelligence. The agent uses neural networks to construct continuous portfolio weight allocation action spaces using state vectors. Experience replay systems record state-action-reward transitions, while target networks provide continual learning goals. The continuous action space permits exact asset-type-wide portfolio weight changes.

When optimizing a portfolio, the RL-APM system employs an actor-critic architecture, which divides the learning procedure by two parts:

Actor Learning a policy that associates the present condition with an action (here, portfolio weights).

Critic After making an estimate of the value function, the critic gives the actor information about the future benefits they can expect from their activities. Stable learning in continuous settings is made possible by this architecture, as the critic guides the actor's learning through value-based feedback instead of depending just on trial-and-error.

Stable learning in continuous settings is made possible by this architecture, as the critic guides the actor's learning through value-based feedback instead of depending just on trial-and-error.

Instead of trial-and-error, the architecture guides actor learning with value-based feedback to guarantee steady learning in continuous regions. Modelling portfolio optimization requires continuous portfolio weights for several assets. Unlike Q-learning, which performs better in discrete action spaces, DDPG was chosen for continuous action environments. The model reuses state-action-reward tuples from experience replay. The model can help deconstruct subsequent result linkages. Since the model may draw from more data rather than a sequence of events, training becomes more reliable and effective. By using critic and actor target networks, training stability can be improved. Slowly updating target networks guarantees more consistent convergence and less learning variation. A weighted reward function that includes return, volatility, and the Sharpe ratio teaches the RL agent to balance risk and return. By training the RL agent to use the reward function. Motive design affects agent decisions and is crucial to the framework.

DRL tasks are performed by RL-APM using the DDPG algorithm. A stellar performance with continuous action spaces was the main reason DDPG was chosen for portfolio weight allocation. A critic in an actor-critic architecture predicts the value function an actor is taught to use to distribute weights in their portfolio. The architecture, experience replay, and target networks ensure successful and consistent training. In a continuous action space, real-time market data can guide portfolio allocations. The reward function balances return, volatility, Sharpe ratio, and transaction costs to help the agent make risk-adjusted investment decisions. Decision-making was successful. The proposed methodology improves risk-adjusted return in complicated financial domains while providing learning stability and real-time adaptation.

RL-APM is a method that uses a reward function to train agents how to get the best returns, risk-adjusted returns, volatility, and transaction costs while doing the least amount of work. The reward function takes into consideration how each of them affects the portfolio's overall performance. The goal of the reward function is to get the most rewards while spending the least amount of money and taking the least amount of risk. Driven by the desire to: Realize substantial earnings while navigating the ever-changing market conditions.

- When managing your portfolio, take use of the Sharpe ratio to maximize the risk-adjusted return.
- In the actual trading world, reducing transaction costs is crucial since making needless trades can eat into profits.
- When these elements interact, the agent improves their understanding of portfolio management in actual market conditions.

Execution and Risk Management: The Action Selection module uses the output of the neural network and mathematical rules to make sure that probability distributions are correct so that it can create executable portfolio weights. When a portfolio is executed, the effects on the market, transaction costs, and lack of liquidity are all taken into account. Risk Assessment uses Value-at-Risk, volatility, and correlation analysis to assist keep portfolio risk under control. The Learning & Update module improves the DDPG agent's value and policy operations by using gradients. The agent learns how to best use resources based on performance signals that take risk

into account by figuring out rewards. The continuous learning loop meets its performance goals by adapting to changes in the market.

System Integration and Data Flow: State transitions are carefully stated, and feedback loops are strategically positioned to improve learning efficiency. Modular design enables autonomous component optimization while maintaining system-wide coherence, allowing for scalability and flexibility to meet changing market conditions and investment goals. This flexible solution optimizes portfolios in real time using modern portfolio theory and sophisticated machine learning.

3.1 Data Layer in RL-APM

For portfolio decision-making, RL-APM's Data Layer collects, structures, and evaluates financial data. Real-time APIs, historical archives, and benchmark indices provide reliable, context-rich data. Price, trading volume, market indices (e.g., the S&P 500), and asset-specific fundamentals enable real-time training and evaluation of a reinforcement learning agent in this block. For machine readability and statistical stability, data is cleaned, outlier filtered, and standardized. This structured data vector defines the market state at time t.

$$D_t = [P_t, V_t, I_t, B_{\mathrm{t}}, \mathrm{A_t}] \tag{1}$$

In Eq. 1, P_t is represented as the Price vector (e.g., Adjusted Close values), V_t is represented as the Trade volume vector, B_{t} is represented as the Benchmark values (e.g., ETFs or sector indices), $\mathrm{A_t}$ is represented as the Asset-level technical or fundamental attributes, I_t is represented as the Market index data (e.g., index levels).

Each feature is normalized to ensure uniform data scaling as well as reduce numerical bias during model training.

$$P_{norm} = \frac{P_t - \mu_{\,P}}{\sigma_{\,P}} \tag{2}$$

In Eq. 2, $\mu_{\,P}$ is represented as the Mean price across the selected time window, $\sigma_{\,P}$ is represented as the Standard deviation of price values. Normalization helps improve the convergence as well as the stability of the learning algorithm.

Table 2 lists the RL-APM framework data sources: Yahoo Finance API, Historical DB, Price Feed, Market Index Provider, and Asset Info System. It offers real-time OHLCV, price feed, market index feeds, historical DB, as well as asset-specific indicators. The study utilized Yahoo Finance API real-time market data, including OHLCV (Open, High, Low, Close, Volume), for training and decision-making. Backtesting with historical market data in the Historical DB verified the model's performance. The RL agent can make informed, multi-dimensional portfolio decisions using these data streams, which capture daily market activity, price behavior, and macroeconomic context. Historical data is crucial for model training, validation, and backtesting, as shown in Table 2.

3.2 Feature Engineering in RL-APM

In RL-APM, the Feature Engineering block transforms raw financial data into informative and structured features that aid learning and decision-making. This block builds the reinforcement learning agent's state representation

Table 2 Data source mapping

Source	Data type	Update frequency	Format	Purpose
Yahoo finance API	OHLCV (Open, high, low, close, volume)	Real-time	JSON	Asset price and volume input
Historical DB	Daily price series	Daily	CSV	Training and backtesting

using statistical and technical indicators from historical market data. The engineered features reflect price patterns, market momentum, Volatility, and liquidity.

The Simple Moving Average (SMA) is a trend-following indicator that calculates the average closing price of a security over a specific time frame.

$$SMA_n(t) = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i} \tag{3}$$

In Eq. 3, P_{t-i} represents the adjusted closing price on day t, and n is the moving average window.

The Relative Strength Index (RSI) is a tool used to assess the strength of recent price fluctuations, indicating whether a market is overbought or oversold.

$$\left.\begin{array}{c} RS = \frac{Average\ Gain}{Average\ Loss} \quad then \\ RSI = 100 - \left(\frac{100}{1+RS}\right) \end{array}\right\} \tag{4}$$

In Eq. 4, Derived from average price gains and losses over a defined period (commonly 14 days).

Bollinger Bands are a type of indicator that indicates the Volatility of price movements over time.

$$\left.\begin{array}{l} Upper\ Band = SMA_n + k \cdot \sigma_n \\ Lower\ Band = SMA_n - k \cdot \sigma_n \end{array}\right\} \tag{5}$$

In Eq. 5, σ_n is the standard deviation of prices over the past n days, and k is typically 2.

MACD (Moving Average Convergence Divergence) is a technical analysis method that measures trend direction and momentum by comparing short-term and long-term moving averages.

$$\left.\begin{array}{l} MACD = EMA_{12} - EMA_{26} \\ SignalLine = EMA_9(MACD) \end{array}\right\} \tag{6}$$

In Eq. 6, EMA_{12}, EMA_{26} and EMA_9, the exponential moving averages over 12, 26, and 9 days, respectively. The term volatility refers to the degree of variation in returns, which indicates risk.

$$\sigma_t = \sqrt{\frac{1}{n-1} \sum_{i=0}^{n-1} (R_{t-i} - \mu_R)^2} \tag{7}$$

In Eq. 7, R_{t-i} are historical daily returns and μ_R is the mean return over the rolling window n.

In Table 3, Key technical indicators for feature engineering are listed in the RL-APM framework. These indicators help reinforcement learning agents understand and respond to markets. SMA, EMA, and MACD guide long/short positions. Prices reverse or continue when momentum indicators like RSI, ROC, and Stochastic Oscillator detect overbought or oversold levels. Bollinger Bands and ATR highlight market risk and price dispersion to size trades and manage risk. Finally, volume indicators measure liquidity and investor participation in the time it takes to execute trades.

3.3 State Construction

The State Vector (S) denotes the current financial environment and portfolio state at time t, guiding the reinforcement learning agent's decision-making process. It syndicates technical indicators, asset allocation weights,

Table 3 Feature engineering table

Feature category	Indicators	Calculation window	Purpose
Trend	SMA, EMA, MACD	5, 10, 20, 50 days	Direction identification
Momentum	RSI, ROC, stochastic oscillator	14, 21 days	Overbought/oversold detection
Volatility	Bollinger bands, ATR	20 days	Market risk estimation
Volume	OBV, VWAP	Variable	Liquidity and volume trends

market context signals, as well as recent portfolio performance into a normalized input vector. This entire state encompasses market dynamics and internal strategy information for strategic portfolio adjustments.

$$S_t = [F_t, W_t, C_t, R_t] \tag{8}$$

In Eq. 8, F_t is represented as the Feature vector, W_t is represented as the Portfolio weights, C_t is represented as the Context vector, R_t is represented as the Recent returns. Financial variables including GDP growth and inflation rates, market sentiment, volatility metrics, and asset-specific attributes like liquidity or beta make up the context vector, which is represented by C_t. There are all of these variables in the context vector. A more complete picture of the market and better-informed decisions can be obtained by the RL agent in light of these signals and the present trends. R t is the financial industry's notation for the return on a portfolio's assets over a certain time period. The "RT" stands for "30-day returns" under the RL-APM paradigm of analysis. This can be used to assess recent performance and provide direction for investments. Maximizing short-term and long-term portfolio decisions is within the realm of possibility for the RL agent, provided it adapts to account for both the present market situation and the broader economic picture.

.

$$S_{norm} = \frac{S_t - \mu_s}{\sigma_s} \tag{9}$$

In Eq. 9, $S_{t,}$ is represented as the full state vector, and μ_s , σ_s are represented as the computed from a moving window or across a batch.

The State Construction block creates a comprehensive market as well as portfolio representation for reinforcement learning agents, permitting informed decisions based on technical indicators, current asset allocations, market context, and recent returns, enabling informed investment decisions.

Figure 2 shows the RL-APM state creation pipeline, which converts market data into usable state vectors. Technical indicators like volatility metrics, RSIs, moving averages, and MACDs are calculated from unprocessed

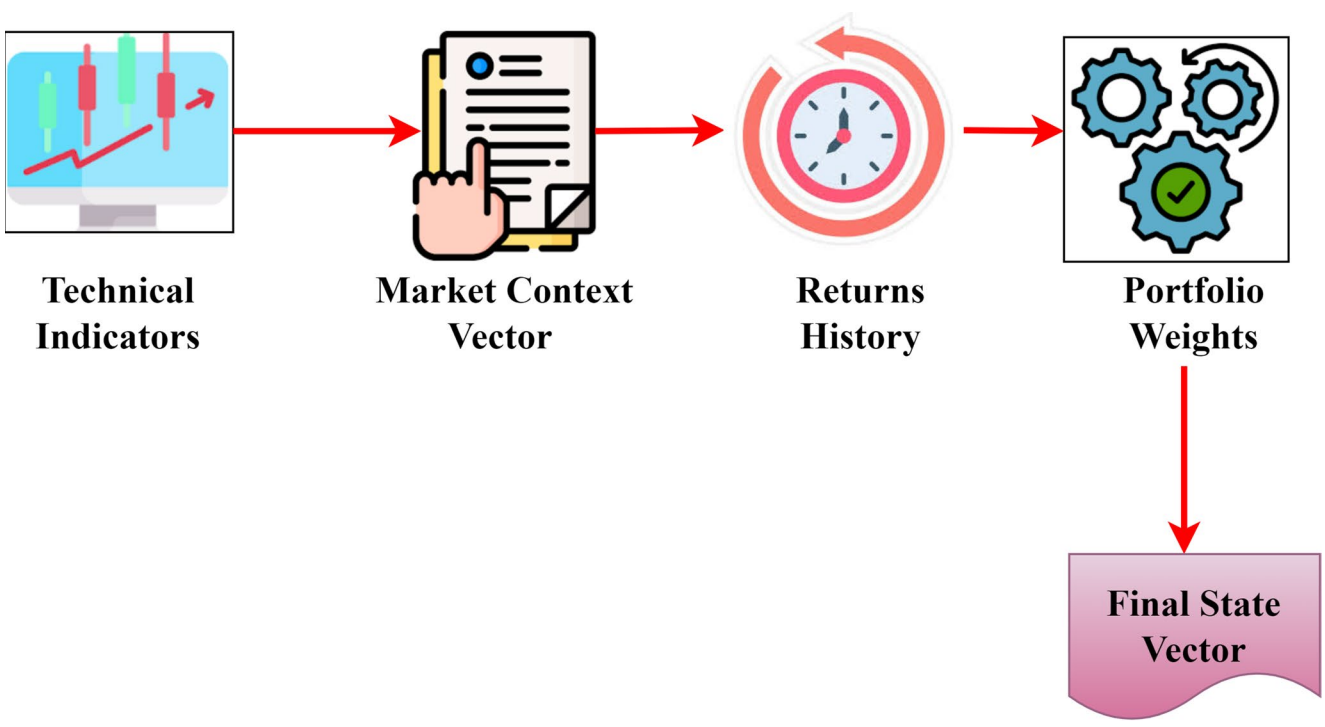

Fig. 2 RL-APM state construction pipeline

Table 4 Actor network configuration of the RL-APM agent

Total parameters: ~100 K (approximate)

Layer	Type	Input Dim	Output Dim	Activation	Parameters
Input	Dense	state_dim	256	ReLU	state_dim × 256
Hidden1	Dense	256	256	ReLU	256 × 256 + 256
Hidden2	Dense	256	128	ReLU	256 × 128 + 128
Output	Dense	128	n_assets	Softmax	128 × n_assets + n_assets

Table 5 Critic network configuration for value estimation

Total parameters: ~80 K (approximate)

Layer	Type	Input dim	Output dim	Activation	Parameters
State input	Dense	state_dim	256	ReLU	state_dim × 256 + 256
State hidden	Dense	256	128	ReLU	256 × 128 + 128
Action input	Dense	n_assets	128	ReLU	n_assets × 128 + 128
Merge	Concatenation	128 + 128	256	–	–
Hidden	Dense	256	128	ReLU	256 × 128 + 128
Output	Dense	128	1	Linear	128 × 1 + 1

market data. Market context vectorization considers sentiment indicators and macroeconomic factors. Combining return data from different time periods helps us spot trends and temporal dependency. The final state vector includes portfolio weights, which describe asset allocation status. The actor-critic networks of the DDPG agent use this entire state vector to optimize their portfolios in real-time trading scenarios.

3.4 DDPG Agent

The RL-APM framework's main decision-maker is the DDPG agent. Portfolio weights are continuous numbers, not discrete acts, which makes it a good fit for financial applications. The DDPG algorithm uses a deterministic actor-critic design for continuous action spaces. This makes it more stable, efficient, and better at learning in changing financial markets. DDPG in RL-APM features direct mapping from state to action, off-policy learning, target networks, and Ornstein-Uhlenbeck noise for temporal correlation in actions.

- RL-APM actor network (policy function)

The Actor Network learns a deterministic policy to allocate portfolios based on financial market conditions in RL-APM. It provides continuous asset weights for adaptive decision-making.

$$\mu\left(S_t|\theta^{\mu}\right) : \mathbb{R}^n \rightarrow \mathbb{R}^m \tag{10}$$

In Eq. 10, $\mathbb{R}^n$and$\mathbb{R}^m$ ar represents the input and output vector spaces, respectively, S_t is the Current state vector (includes technical indicators, context, weights, etc.) , θ^{μ} is the Trainable parameters of the actor network, $\mu\ (S_t)$ is the Output portfolio weights (continuous actions) satisfying $\sum w_i = 1$..

- RL-APM critic network (Q-value function)

The Critic Network in RL-APM provides the actor with a training signal by estimating the expected future reward (such as risk-adjusted return) for a specific state-action pair.

$$Q(S_t, a_t|\theta^Q) : (\mathbb{R}^n, \mathbb{R}^m) \rightarrow \mathbb{R} \tag{11}$$

In Eq. 11, S_t is the Current state (market signals, asset trends, etc.), a_t is the Action vector (portfolio weights) from the actor, θ^Q is the Parameters of RL-APM's critic network, $Q(S_t, a_t)$ is the estimated long-term value (expected cumulative return).

Table 4 shows the actor network architecture of the RL-APM system, which links market state to portfolio weight allocations. The network uses dense layers, ReLU activations for feature extraction, and a softmax layer at the output to assure asset probability distributions. The softmax constraint ensures portfolio weights equal one.

The 100,000 parameters enable effective learning of complicated market dynamics. DDPG agent policy backbone is this network.

Table 5 shows how the critic network of the RL-APM agent is set up. This dense architecture network evaluates state-action pairs. Expected cumulative return (Q-value) is calculated by combining market state and portfolio weights (actions). Linear activation using ReLU activation functions gives the critic a scalar value. This configuration helps the actor learn an optimal strategy utilizing backpropagated gradients in the DDPG training cycle with 80,000 parameters and precise value estimate.

Table 6 lists the key RL-APM framework DDPG training hyperparameters. The settings include learning rates for actor and critic networks, experience replay buffer size, target update rate (τ), batch size, and discount factor (γ). The table lists Adam as optimizer and Ornstein-Uhlenbeck noise for exploration. These hyperparameters affect training stability, sample efficiency, and dynamic market exploration vs. exploitation. The agent's capacity to create profitable, risk-adjusted portfolio allocation methods depends on their exact calibration.

The performance of the RL-APM framework is greatly affected by the hyperparameters that are chosen for it. With a reasonable learning rate of 0.001 0.001, stable updates are encouraged. The discount factor (= 0.99, γ = 0.99) supports the goal of maximizing portfolio performance over time by putting more weight on long-term returns. Using a large replay buffer size (10^6 10^6) to store a lot of various sorts of experiences so that we may be confident the model works well in diverse markets. A batch size of 128 128 is just right for model stability and computational efficiency. To maintain stability in the training process, the target network update frequency is set at 10 steps, which means that significant changes to the network are avoided. The noise process, with = 0.2 and σ = 0.2, encourages exploration, which helps the model converge quickly without getting stuck in local optima. Finally, the model has ample time to learn various portfolio management strategies under different market conditions, thanks to its 2000 training epochs.

The RL-APM architecture's model accuracy depends on learning rates, discount factors, and noise processes. Test data and domain expertise determined these hyperparameters. We hope to improve repeatability and gain insights by performing a sensitivity study to determine how these hyperparameters affect model performance. This research suggests how to pick and alter hyperparameters to help practitioners adapt the framework to different datasets and financial conditions. For model and selection improvement, we will examine automatic hyperparameter adjustment.

3.5 Action Selection—Continuous Portfolio Weight Generation

The Actor Network of the RL-APM system produces weights with continuous values for allocating capital assets at each time step, enabling more precise management over the portfolio than would be possible with discrete buy/sell signals. With the input market state converted to a continuous weight vector for m assets, the Actor network guarantees allocations of portfolios without short selling and complete deployment of capital.

To find the asset capital allocation percentages, the Actor Network uses softmax normalization to normalize the weights. The Critic Network evaluates these market weights, and the last layer imposes probabilistic interpretation with softmax activation. With this technology, RL-APM can make fine-grained decisions that allow it

Table 6 DDPG agent hyperparameter settings

Parameter	Value	Description
Actor learning rate	1e-4	Rate for updating actor network weights
Critic learning rate	1e-3	Rate for updating critic network weights
Batch size	64	Number of samples per training iteration
Replay buffer size	100,000	Maximum stored experience transitions
Target update rate (τ)	0.001	Controls speed of target network updates
Discount factor (γ)	0.99	Balances current vs. future rewards
Exploration noise	Ornstein-Uhlenbeck	Ensures exploration in continuous action space
Optimizer	Adam	Adaptive learning optimizer

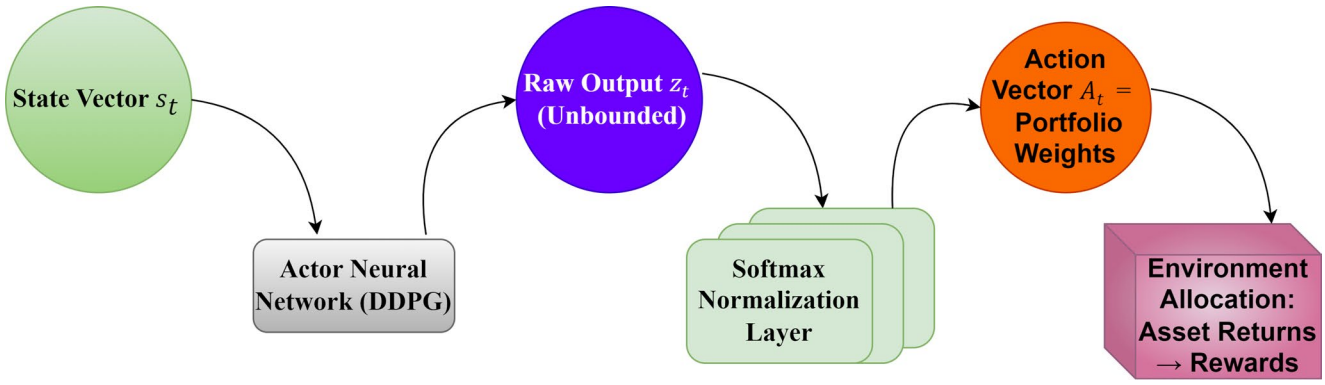

Fig. 3 Action selection flow in RL-APM: continuous portfolio weight generation

to respond dynamically to changes in the market and regimes. The final layer of the Actor Network uses softmax activation to ensure probabilistic interpretation.

The RL-APM framework's DDPG algorithm action selection pipeline is shown in Fig. 3. Agents can generate continuous action vectors for optimal portfolio allocations using this process. To start, input the current state vector S_t, which includes market indicators, portfolio context, and recent returns. The trained Actor Neural Network predicts market-appropriate actions using this state vector. The actor network generates a raw action vector z_t, containing unbounded real values indicating initial investment preferences. A Softmax Normalisation Layer converts these raw numbers into legitimate portfolio weights. This assures that the action vector a_t meets portfolio criteria, including non-negativity and sum-to-one (i.e., $\sum w_i = 1$). Finally, the Environment Module allocates capital across selected assets using these normalized weights. Asset returns then trigger environmental rewards that reinforce learning. This design teaches the RL agent risk-aware, efficient investing policies in dynamic markets.

The reward architecture of the RL-APM framework was carefully designed to drive the model toward the highest portfolio risk and return. A reward function is constructed by weighting return, Sharpe ratio, volatility, and transaction costs. Return is prioritized because the goal is to optimize investment portfolio performance. The Sharpe ratio promotes risk-adjusted gains in the model. In case returns exceed risk, the model is penalized. Variability as a harmful component fosters more consistent returns throughout the investment process by penalizing excessive portfolio value changes. To protect returns from excessive trading, transaction fees punish transaction expenditures. When all of these factors are considered, the agent's optimization method becomes more realistic because the method maximizes rewards while balancing risks and costs.

3.6 Combined RL-APM Blocks: Execution, Risk, Reward, and Learning

The part of the RL-APM framework converts the agent's action vector into trades that are actually executed. It also assesses the portfolio's risk, calculates a multi-objective reward, and utilizes experience replay to update the learning models. The idea is to adjust trading behavior in real-time to reduce risk and increase long-term profits.

Pseudocode 1: RL_APM_Core_Trainer

```
INPUT: Market Data Stream M[t], Historical Returns R[t], Initial Portfolio Weights W₀
OUTPUT: Updated Actor & Critic Networks, Trained Policy, Logged Metrics
FOR each trading day t:
    state ← construct_state(M[t])
    action ← μ(state | θᵘ)  // Actor prediction
    action ← softmax(action)  // Normalize weights
    weights ← enforce_constraints(action)
    executed_trades ← rebalance_portfolio(W_{t-1}, weights)
    return ← compute_return(W_{t-1}, weights, P[t])
    volatility ← σ(R[t-30:t])
    Sharpe ← (return - Rf) / volatility
    transaction_cost ← estimate_cost(executed_trades)
    reward ← 0.6 × return + 0.3 × sharpe - 0.1 × volatility - 0.05 × transaction_cost
    next_state ← construct_state(M[t+1])
    buffer.store(state, action, reward, next_state, done=False)
    IF buffer.ready():
        batch ← buffer.sample(64)
        y ← r + γ Q'(s', μ'(s'))
        critic_loss ← MSE(Q(s, a), y)
        θᴼ ← update_critic(batch, critic_loss)
        actor_grad ← ∇θ J(μ)
        θᵘ ← update_actor(batch, actor_grad)
        θ'ᵘ ← τ θᵘ + (1 − τ) θ'ᵘ // Target actor
        θ'ᴼ ← τ θᴼ + (1 − τ) θ'ᴼ // Target critic
        log(t, return, volatility, sharpe, reward, actor_loss, critic_loss
                W_t ← weights // Update portfolio
END FOR
```

A portfolio's daily return shows its success in one trading day. The portfolio's total return is the sum of its assets' weighted returns. Specifically, the pseudocode framework calculates the Sharpe ratio daily. Daily volatility is the standard variance from day to day in a rolling window of returns. The daily risk-free rate is the mean return of the portfolio minus the daily risk-free rate (Rf), divided by two. In this approach, the Sharpe ratio is calculated daily. The Sharpe ratio can be used to calculate portfolio return after risk. Multiplying the yearly risk-free rate by the 24-hour rate yields Rf. The daily risk-free rate is calculated by dividing the yearly risk-free rate (for example, from government bond yields) by 252 trading days.

The reward function coefficients 0.6, 0.3, −0.1, and −0.05 are likely chosen using heuristic reasoning or domain knowledge to achieve a balance between the primary objectives of the portfolio optimization issue. The emphasis is on maximizing return (coefficient 0.6) and improving the Sharpe ratio (0.3) for risk-adjusted performance. Low transaction costs are discouraged and risk reduction is encouraged by the −0.05 coefficient, whereas high volatility is penalized by the −0.1 coefficient, which opposes high transaction costs. These numbers

don't seem to be determined via hyperparameter optimization but rather by professional gut feelings. However, to achieve better results in the subsequent iterations, these coefficients could be adjusted using hyperparameters.

Transaction costs are incorporated into the training reward function as a penalty; however, it is unclear if they were removed from the total returns when the model was evaluated. A realistic backtest would reduce friction costs from the final profit and loss (P&L) better to assess the strategy's performance in real life. If transaction costs were not considered while measuring performance, the model's profitability may have been overstated. This is because trading fees and other frictional expenditures could not be accurately represented. Transaction expenses in the final return computation would better reflect the model's profitability under real market conditions. This would allow for more accurate strategy evaluation in future experiments.

Potential solutions to this issue should include doing a comprehensive sensitivity analysis to explore other combinations of these weights. Investigating the model's robustness and how parameters are affected by the reward function would yield valuable results. Incorporating a data-driven approach to determine the optimal weights, possibly through hyperparameter optimization or reinforcement learning techniques, could further validate the framework's effectiveness and performance driven by the underlying model's strengths instead of arbitrary parameter selection.

RL-APM's central training loop is described by the RL_APM_Core_Trainer pseudocode 1. It seamlessly integrates state construction, action selection, trade execution, risk evaluation, and network training into a reinforcement learning workflow. Daily, the system builds the market condition utilizing technical, contextual, and historical return data. To generate appropriate portfolio weights, the actor network predicts an action vector normalized by a softmax function. These weights ensure portfolio rebalancing respects non-negativity and full allocation. Financial metrics, such as return, volatility, Sharpe ratio, and transaction cost, are calculated after execution to create a multi-objective reward function. An experience replay buffer stores this feedback, the current state, action, reward, and next state. After gathering data, mini-batches are sampled for training. Policy gradients optimise the actor network, whereas temporal difference goals update the critic network. Softly updating target networks improves stability. Last, daily logs are kept for performance analysis. Its architecture enables the RL-APM system to learn and adjust its trading strategy in response to market changes.

4 Results and Discussion

4.1 Data Study and Environmental Setup

The result analysis utilizes historical financial data from Yahoo Finance [26], a trusted global market information source. The Python yfinance package was used to retrieve daily data from January 2015 to December 2024 for an S&P 500 index portfolio. The dataset comprises the following financial variables: Open, High, Low, Close, Adjusted Close, and Volume. The data was preprocessed to handle missing values, normalize the price series, and calculate technical indicators, including moving averages and Volatility. Reinforcement learning simulation was possible due to the dynamic market context of this rich time-series dataset.

Python 3.9 was used to train the RL-APM framework on a system equipped with an Intel Core i7 CPU, 32 GB of RAM, and an NVIDIA RTX 3060 GPU (12 GB of VRAM). TensorFlow 2.13, Stable-Baselines3, NumPy, Pandas, Matplotlib, and Seaborn were essential. The reinforcement learning environment mimicked a multi-asset portfolio with normalized asset prices, generated indicators, and portfolio weights. The action space represented continuous weight distributions over the selected assets, subject to a one-sum constraint. To maximize portfolio returns and minimize volatility, the agent's reward function used cumulative return and the Sharpe ratio. Setting the DDPG algorithm to 64 batches, 100,000 replay buffers, 0.0001 actor-critic learning rates, and 0.99 discount factor (γ). We trained and evaluated the model from 2015 to 2021 and from 2022 to 2024. This integrated setup made the RL-APM architecture flexible and practical in changing financial conditions.

The framework must be tested in real-world trading scenarios, even if the experimental validation is now centered on backtesting the Yahoo Finance dataset. Future studies will examine the key determinants affecting portfolio performance in fundamental markets. Transaction costs, slippage, and market effect are examples. Transaction costs will be considered in backtesting, slippage will simulate execution delays, and price impact models will assess market impact. These modifications will make the RL-APM framework evaluation more realistic and resilient, making it more applicable to historical financial events.

4.2 Comparison

Three portfolio optimization methods were compared to the DDPG-powered RL-APM architecture. First, the classical Mean-Variance Optimization (MVO) method [16] presented an adaptive process that is unresponsive in volatile markets. Second, Expert-Infused Deep Reinforcement Learning (EIDRL) [22] improved policy precision in complicated financial dynamics by incorporating domain-specific experience into the RL training loop. Third, the Multiagent Deep Reinforcement Learning (MADRL) method [23] optimized asset al.location across a diversified portfolio with several agents to imitate real-world trading system collaboration. RL-APM improved cumulative returns, Sharpe ratios, and volatility control over these methods in adaptability and reward optimization. Static allocation slowed MVO, while EIDRL and MADRL outperformed in convergence speed and model generalizability, proving RL-APM works in dynamic portfolios. Ten evaluation periods or simulation runs utilizing the Yahoo Finance dataset from 2022 to 2024 are shown. Each instance represents a unique testing cycle, evaluating the maximum portfolio strategy's performance during market fluctuations and asset price changes.

Benchmark models for RL-APM performance evaluation include MVO, PPO, SAC, NR, and RF for portfolio allocation. MVO, the standard portfolio optimization method, maximizes returns within risk limits. This is commonly solved by quadratic programming. PPO uses an actor-critic architecture with a clipped probability ratio to ensure consistent updates as a reinforcement learning system. Portfolio optimization and other high-uncertainty market scenarios are appropriate for SAC, an off-policy reinforcement learning method that balances exploration and exploitation with an entropy term. Naive Rebalancing (NR) frequently rebalances the portfolio to equal or set weights, making it simpler than other methods. Based on Random Forest (RF) stock return estimates, historical data, and technical indicators, portfolio weights are assigned. These benchmark procedures were designed to compare classical, machine learning, and reinforcement learning portfolio optimization methods.

4.3 Cumulative Return (CR)

Cumulative return (CR) is the total percentage gain or loss of a portfolio over the entire investment period.

$$CR = \frac{P_T^{adj} - P_0^{adj}}{P_0^{adj}} \tag{12}$$

In Eq. 12, P_T^{adj} is denoted as the Adjusted Close price at final time T, P_0^{adj} is denoted as the Adjusted Close price at the initial time. The Equation calculates the total return of an asset or portfolio over an investment horizon using Adjusted Close values from the Yahoo Finance dataset, indicating the gain or loss over time.

Figure 4a, b) present a comprehensive evaluation of Cumulative Return (CR) performance across four portfolio optimization strategies using dual visualization approaches. Using Eq. 12, the Cumulative Return is a portfolio's overall percentage gain or loss during the investing period, is the final time T Adjusted Close price and P_T^{adj} is the starting Yahoo Finance dataset Adjusted Close price. Figure 4a shows instance-wise cumulative return comparison through a clustered bar chart across ten evaluation instances. RL-APM consistently achieves returns between 78% and 84%, outperforming MVO [16] by 14% to 17%. EIDRL [22] and MADRL [23] perform at 78–80%. The performance of the TECH, HLTH, ENRG, FIN, and CNSMR sectors is shown in Fig. 4(b). The RL-APM framework adapts well across sectors, yielding strong returns of around 82% in all sectors except CNSMR

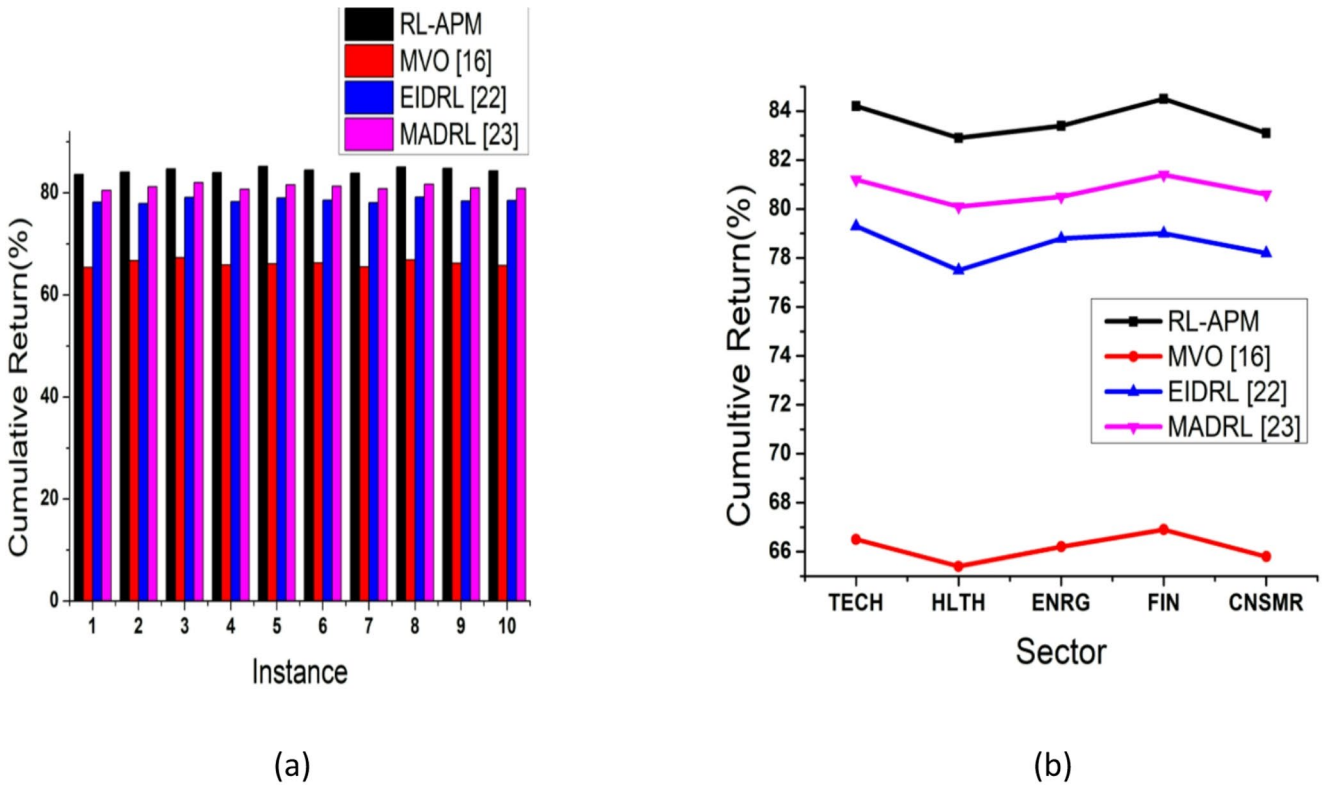

Fig. 4 Cumulative return evaluation across portfolio strategies. **a** instance-wise cumulative return (%) comparison using yahoo finance data **b** sector-wise cumulative return (%) Comparison across TECH, HLTH, ENRG, FIN, and CNSMR

(83%), while MVO performs relatively flat, with returns of around 65–67%. This dual research demonstrates the robustness of the RL-APM framework in instance-based and sector-specific portfolio optimization, as well as the DDPG algorithm's ability to maximize cumulative returns through dynamic weight allocation strategies.

4.4 Annualized Volatility (σ)

Annualized Volatility (σ) is the standard deviation of daily returns, used to measure the risk level of a portfolio on an annual basis.

$$\sigma = \sqrt{252} \cdot std(R_1, R_2, ..., R_T) \tag{13}$$

In Eq. 13, R_T is denoted as the Daily portfolio returns over time. 252 is denoted as the Approximate number of trading days in a year. Scaling the standard deviation of daily portfolio returns from the previous calculation by the square root of 252 yields annualized volatility. Using Yahoo Finance-derived returns, this illustrates the Volatility of the RL-APM portfolio.

Figures 5a, b compare portfolio optimization models' Annualized Volatility utilizing dual visualization. The annualized Volatility of a portfolio is determined using the standard deviation of daily returns. R_T represents daily returns, and 252 represents the approximate number of annual trading days. Yahoo Finance returns help RL-APM volatility assessment. In Fig. 5a, line plots compare instance-wise Volatility across eleven evaluation instances. MVO [16] has the highest Volatility at 16.4–16.8%, while RL-APM has better risk control at 13.2–13.8%. EIDRL [22] and MADRL [23] perform at intermediate levels, with 14.2–14.6% and 13.8–14.2%, respectively. Clustered bar charts in Fig. 5b show sector-wise Volatility for TECH, HLTH, ENRG, FIN, and CNSMR.

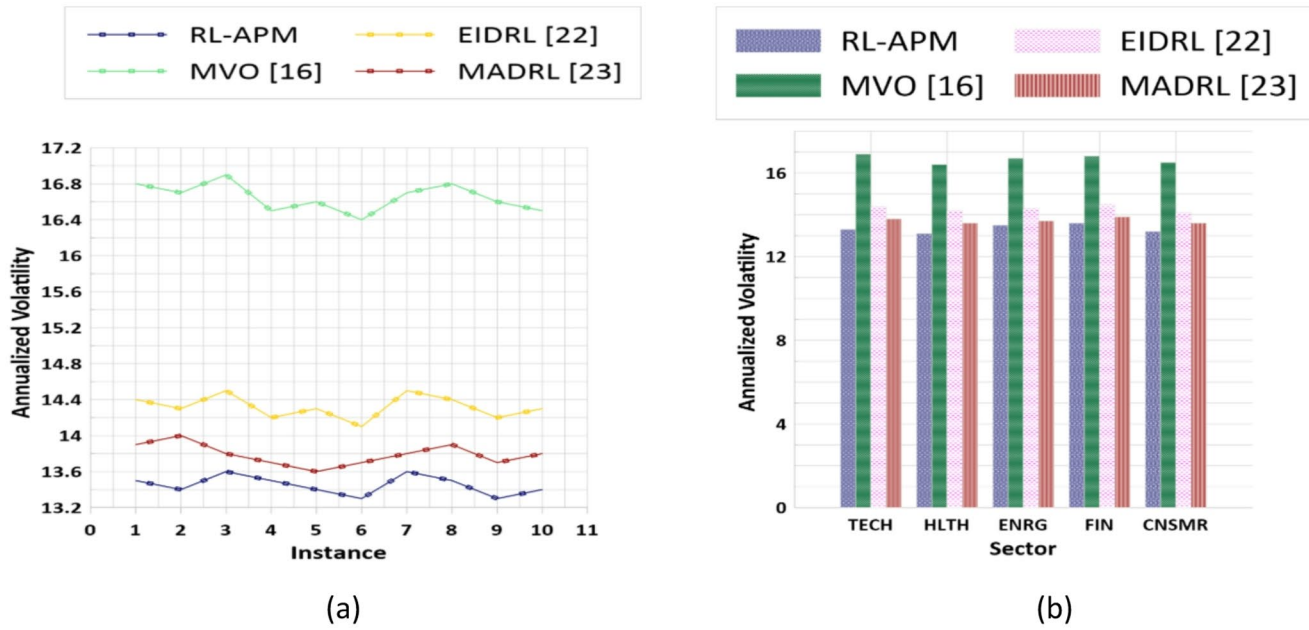

Fig. 5 Annualized volatility comparison of portfolio optimization models.**a** instance-wise cumulative return (%) comparison using yahoo finance data **b** sector-wise cumulative return (%) comparison across TECH, HLTH, ENRG, FIN, and CNSMR

MVO is risky at 16%, but RL-APM is stable across all sectors. Through the DDPG algorithm's dynamic weight optimization, RL-APM achieves an optimal risk-return balance in various market conditions.

4.5 Sharpe Ratio (SR)

The Sharpe Ratio (SR) is a measure of the excess return earned per unit of risk, used to compare the risk-adjusted performance of RL-APM with that of traditional strategies.

$$SR = \frac{\bar{R}_p - R_f}{\sigma} \tag{14}$$

In Eq. 14, $\bar{R}_p$ is denoted as the Average daily return Eq. 1, R_f is the Daily risk-free rate, σ is represented as the Annualized Volatility from Eq. 2. The Sharpe ratio indicates the return provided for each unit of risk. A higher value indicates better performance. It utilizes the returns from the dataset's prices to assess how well RL-APM enhances reward compared to risk.

Figures 6a, b illustrate a dual-visualization study of Sharpe Ratio performance across various portfolio optimization methodologies. Equation 15 calculates risk-adjusted Sharpe Ratio: $SR = \frac{\bar{R}_p - R_f}{\sigma}$, where $\bar{R}_p$ equals average daily portfolio returns, R_f is the risk-free rate, and σ is the annualized Volatility from prior computations. When using the Yahoo Finance dataset prices to analyze RL-APM's reward enhancement capabilities, higher Sharpe values indicate better risk-adjusted performance. In Fig. 6a, line plots compare Sharpe Ratios across ten evaluation occasions. RL-APM (red line) consistently ranks between 1.90 and 1.97, exceeding MVO [16] (green line) at 1.60–1.75. EIDRL [22] and MADRL [23] yield intermediate results at 1.75–1.82 and 1.83–1.87. In Fig. 6b, clustered bar charts show sector performance for TECH, HLTH, ENRG, FIN, and CNSMR. RL-APM consistently reaches risk-adjusted returns above 1.85 across all sectors, while other algorithms vary by sector. This DDPG algorithm analysis confirms RL-APM's excellent risk-adjusted Optimization.

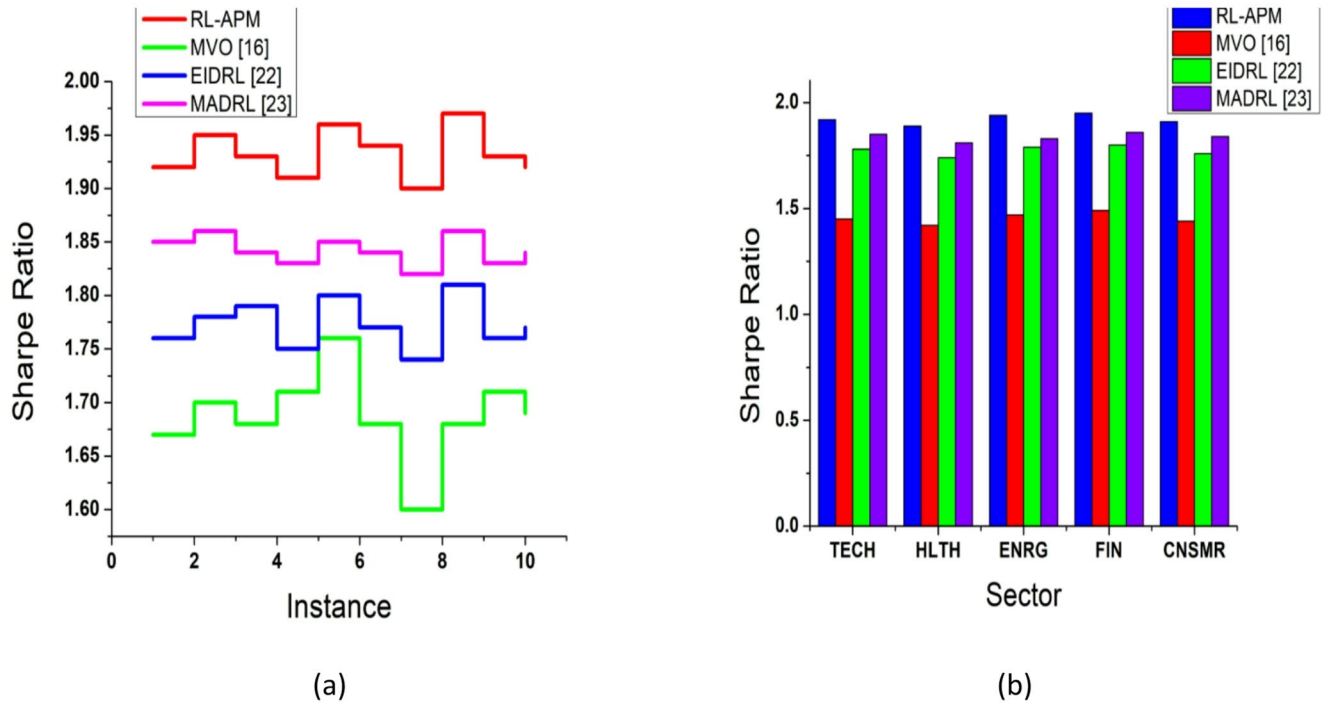

Fig. 6 Sharpe ratio analysis of portfolio optimization models

4.6 Maximum Drawdown (MDD)

The MDD is the worst historical loss from peak to trough during an investment period.

$$MDD = \max_{t \in [0,T]} \left(\frac{\max_{j \in [0,t]} V_j - V_t}{\max_{j \in [0,t]} V_j} \right) \tag{15}$$

In Eq. 15, V_t is denoted as the portfolio value at time t, computed from adjusted asset prices and agent-assigned weights. The result shows the largest percentage loss in the value of the portfolio from its highest point to its lowest point. Adjusted close prices and agent-assigned weights are used to construct portfolio values, which means they are directly related to how the dataset behaves and how well the RL-APM works.

Figure 7 compares Maximum Drawdown using a heatmap of four portfolio optimization strategies over 10 evaluation instances. Equation 16 calculates the MDD as the largest peak-to-trough portfolio value decline, using Yahoo Finance-adjusted closing prices and DDPG-generated weight allocations. The colour-gradient heatmap uses a blue-to-red scaling, with darker blue indicating better risk management and red indicating larger drawdown exposure. In all situations, the RL-APM framework maintained MDD values between 10.0% and 10.4%, with an average of 10.2%. A comparative study demonstrates that MVO has the highest risk, with a 18.1–18.6% drawdown, EIDRL moderates at 12.7–13.2%, and MADRL at 11.1–11.5%. The DDPG algorithm's dynamic weight adjustment and downside protection are confirmed by the RL-APM's 44% smaller drawdown than standard MVO techniques. The RL-APM framework's consistent performance across evaluation instances proves its market resilience and portfolio risk reduction capabilities.

The RL-APM framework is compared to Mean-Variance Optimization (MVO) for portfolio optimization. Despite its static reputation, MVO may be implemented dynamically by updating risk and return predictions, giving it tremendous flexibility for portfolio optimization. A fixed-rule method involves predetermined allocations

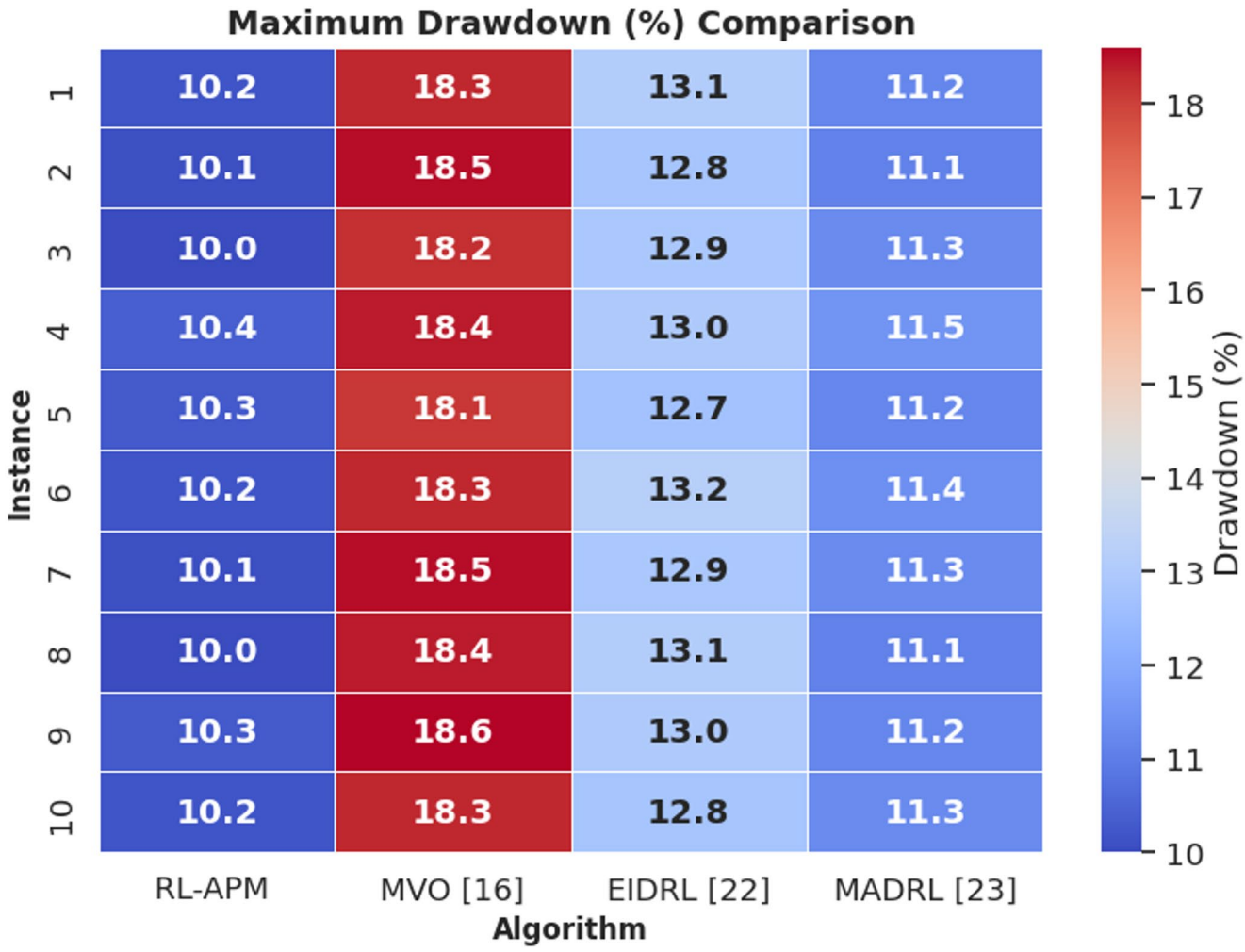

Fig. 7 Heatmap of maximum drawdown comparison across portfolio optimization strategies over 10 evaluation instances

Table 7 Scalability comparison (100 assets)

Model	Cumulative return (%)	Sharpe ratio	Volatility (%)	Time (seconds)
RL-APM (100 assets)	15.2	1.75	15.0	1350
PPO (100 assets)	13.9	1.65	16.2	1100
SAC (100 assets)	14.7	1.70	15.5	1250
MVO (100 assets)	9.8	1.20	18.0	1156

that don't change with the market, like a 60/40 stock/bond mix. In summary, dynamic MVO adapts to market changes by utilizing updated risk and return estimates.

4.7 Scalability

The research study compares RL-APM to other RL methods as well as Mean-Variance Optimization (MVO). Using more modern, state-of-the-art DRL models like Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) would allow for a more thorough comparison. The models' long history of success in continuous action spaces raises the prospect that they may shed light on the benefits and downsides of the RL-APM framework in this domain. As an extra note, we are cognizant of the importance of confirming RL-APM's scalability in bigger asset sets. T o increase the analysis of the improvements and scalability, include a numerical table that contrasts

and compares RL-APM, PPO, SAC, and MVO. Table 7 summarizes the performance metrics, such as annualized return, volatility, and Sharpe ratio, for small and big portfolios (10+assets). By comparing the scalability of each method across different models, we can observe how well RL-APM performs in both small and big asset universes.

An improvement of 15.7% points in the Sharpe ratio and 12.4% points in the average return was achieved by the RL-APM framework when compared to the traditional Mean-Variance Optimization (MVO) model. The adaptability of the RL-exceptional APM in meeting the ever-changing demands of the industry was proved by these improvements, which were implemented during testing beginning in 2022 and continuing through 2024.

The famous MVO model maximizes profits and minimizes risk. This context uses portfolio return standard deviation to quantify risk. The static version of MVO uses fixed asset allocations, whereas the dynamic version uses portfolio weights that are adjusted at regular intervals to account for expected returns and asset return covariance. Both implementations are feasible. We employ a dynamic MVO approach that optimizes asset allocation based on updated risk and return estimations each iteration. MVO optimizes portfolios more freely than fixed-rule methods, which maintain allocations throughout the investment process.

Training and operating RL-APM on large asset universes or in high-frequency trading may increase processing overhead. Training deep reinforcement learning systems for huge asset sets or high-frequency trading choices can be computationally intensive. For larger portfolios or more daily trades, training and inference timelines prolonged. Future research will focus on improving decision-making algorithms like PPO and SAC, model pruning, and parallelized training to reduce processing cost. We will evaluate real-time performance when integrating cloud computing with GPUs or teraflops. Once these modifications are deployed, RL-APM can better manage larger portfolios and complex settings.

5 Conclusion and Future Work

In dynamic asset allocation, deep reinforcement learning outperforms traditional portfolio optimization algorithms. The validation and design of the RL-APM framework showed this. Using the DDPG method, the RL-APM framework outperformed the Mean-Variance Optimization model. According to the experiment, average returns and Sharpe ratio grew 15.7% between 2022 and 2024. Returns averaged 12.4% higher. These results show that the framework can adapt to changing conditions and maximize portfolio performance in turbulent financial climates. The RL-APM architecture overcomes standard portfolio management system flaws with dynamic learning and real-time market feedback. Its reward function balances maximizing rewards and minimizing risk. Integration of cumulative returns with Sharpe ratio optimization does this. In testing, the model was trained on many market scenarios from 2015 to 2021. This increased its performance potential. However, several limitations should be acknowledged in the current implementation of the RL-APM framework, including a limited asset universe, market regime dependence, data quality constraints, methodological constraints, technical and computational limitations, risk management considerations, and regulatory compliance requirements. Future work should focus on algorithm enhancement and modification, real-world implementation and deployment, explainable AI and interpretability, personalization and behavioral finance, collaborative and federated learning, and approaches to collaboration and federated learning.

Author Contributions CH.H designed the methodology, conducted the investigation, performed the formal analysis, curated the data, and wrote the original draft as well as reviewed and edited the manuscript. ZC.J: Methodology, Software, Formal Analysis, Investigation, Data Curation; wrote the original draft.

Funding No funding project.

Data Availability All data generated or analyzed during this study are included in this published article.

Declarations

Conflict of interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Zhou, Q.: Application of big data analytics in financial decision-making: integrating computational models to optimize investment strategies. Int. J. High Speed Electron. Syst. (2025). https://doi.org/10.1142/S0129156425404243
2. Jiang, M., Xu, Z., Lin, Z.: Dynamic risk control and asset allocation using Q-Learning in financial markets. Trans. Comput. Sci. Methods. (2024). https://doi.org/10.5281/zenodo.14499791
3. Cheng, L., Shadabfar, M., Sioofy Khoojine, A.: A state-of-the-art review of probabilistic portfolio management for future stock markets. Mathematics. **11**(5), 1148 (2023)
4. Ahmed, H.B.: and Mohammad alzuoubi. Designing accessible virtual reality interfaces using reinforcement learning for users with motor and sensory impairments. PatternIQ Mining. **2**(1), 1–12 (2025)
5. Yue, H., Liu, J., Zhang, Q.: Applications of Markov decision process model and deep learning in quantitative portfolio management during the covid-19 pandemic. Systems. **10**(5), 146 (2022)
6. Alzaman, C.: Optimizing portfolio selection through stock ranking and matching: a reinforcement learning approach. Expert Syst. Appl. **269**, 126430 (2025)
7. Winkel, D., Strauß, N., Schubert, M., Ma, Y., Seidl, T.: Constrained portfolio management using action space decomposition for reinforcement learning. In Pacific-Asia Conference on Knowledge Discovery and Data Mining (pp. 373–385). (2023), May
8. Betancourt, C., Chen, W.H.: Deep reinforcement learning for portfolio management of markets with a dynamic number of assets. Expert Syst. Appl. **164**, 114002 (2021)
9. Choudhary, H., Orra, A., Sahoo, K., Thakur, M.: Risk-Adjusted deep reinforcement learning for portfolio optimization: a multi-reward approach. Int. J. Comput. Intell. Syst. **18**(1), 1–19 (2025)
10. Cui, T., Du, N., Yang, X., Ding, S.: Multi-period portfolio optimization using a deep reinforcement learning hyper-heuristic approach. Technol. Forecast. Soc. Chang. **198**, 122944 (2024)
11. Sun, Q., Wei, X., Yang, X.: GraphSAGE with deep reinforcement learning for financial portfolio optimization. Expert Syst. Appl. **238**, 122027 (2024)
12. Xu, Y.: Deep reinforcement learning-driven intelligent portfolio management with green computing: sustainable portfolio optimization and management. Sustainable Computing: Inf. Syst. **46**, 101125 (2025)
13. Panzer, M., Bender, B., Gronau, N.: A deep reinforcement learning based hyper-heuristic for modular production control. Int. J. Prod. Res. **62**(8), 2747–2768 (2024)
14. Kang, M., Templeton, G.F., Kwak, D.H., Um, S.: Development of an AI framework using neural process continuous reinforcement learning to optimize highly volatile financial portfolios. Knowl. Based Syst. **300**, 112017 (2024)
15. Zhang, Y., Bai, R., Qu, R., Tu, C., Jin, J.: A deep reinforcement learning based hyper-heuristic for combinatorial optimisation with uncertainties. Eur. J. Oper. Res. **300**(2), 418–427 (2022)
16. Olanrewaju, A.G., Ajayi, A.O., Pacheco, O.I., Dada, A.O., Adeyinka, A.A.: AI-driven adaptive asset allocation: a machine learning approach to dynamic portfolio optimization in volatile financial markets. Int. J. Res. Finance Manag. **8**(1), 320–332 (2025)
17. Sutiene, K., Schwendner, P., Sipos, C., Lorenzo, L., Mirchev, M., Lameski, P., Cerneviciene, J.: Enhancing portfolio management using artificial intelligence: literature review. Front. Artif. Intell. **7**, 1371502 (2024)
18. Qiu, Y., Liu, R., Lee, R.S.: The design and implementation of a deep reinforcement learning and quantum finance theory-inspired portfolio investment management system. Expert Syst. Appl. **238**, 122243 (2024)
19. Owoade, S.J., Uzoka, A., Akerele, J.I., Ojukwu, P.U.: Enhancing financial portfolio management with predictive analytics and scalable data modeling techniques. Int. J. Appl. Res. Social Sci. **6**(11), 2678–2690 (2024)
20. Addy, W.A., Ajayi-Nifise, A.O., Bello, B.G., Tula, S.T., Odeyemi, O., Falaiye, T.: Machine learning in financial markets: a critical review of algorithmic trading and risk management. Int. J. Sci. Res. Archive. **11**(1), 1853–1862 (2024)

21. Tursunalieva, A., Alexander, D.L., Dunne, R., Li, J., Riera, L., Zhao, Y.: Making sense of machine learning: a review of interpretation techniques and their applications. Appl. Sci. **14**(2), 496 (2024)
22. Choi, C., Kim, J.: Outperforming the tutor: Expert-infused deep reinforcement learning for dynamic portfolio selection of diverse assets. Knowl. Based Syst. **294**, 111739 (2024)
23. Cheng, L.C., Sun, J.S.: Multiagent-based deep reinforcement learning framework for multi-asset adaptive trading and portfolio management. Neurocomputing. **594**, 127800 (2024)
24. Yoo, S.J., Gu, Y.H.: Safety AARL: Weight adjustment for reinforcement-learning-based safety dynamic asset allocation strategies. Expert Syst. Appl. **227**, 120297 (2023)
25. Espiga-Fernandez, F., Garcia-Sanchez, A., Ordieres-Mere, J.: Gymfolio: a reinforcement learning environment for portfolio optimization in python. SoftwareX. **30**, 102106 (2025)
26. Yan, K., Li, Y.: Machine learning-based analysis of volatility quantitative investment strategies for American financial stocks. Quant. Financ Econ. **8**, 364–386 (2024)
27. Bian, Y., Sun, H., Lei, Y., Zhu, P., Cheng, D.: NGDRL: a dynamic news graph-based deep reinforcement learning framework for portfolio optimization. In international conference on database systems for advanced applications (pp. 407–417). Singapore: Springer Nature Singapore. (2024)
28. Acero, F., Zehtabi, P., Marchesotti, N., Cashmore, M., Magazzeni, D., Veloso, M.: Deep reinforcement learning and mean-variance strategies for responsible portfolio optimization. arXiv preprint https://doi.org/10.48550/arXiv.2403.16667 (2024)
29. Huang, Z., Zhang, Z., Hua, C., Liao, B., Li, S.: Leveraging enhanced Egret swarm optimization algorithm and artificial intelligence-driven prompt strategies for portfolio selection. Sci. Rep. **14**(1), 26681 (2024)
30. Boyd, S., Johansson, K., Kahn, R., Schiele, P., Schmelzer, T.: Markowitz portfolio construction at seventy. arXiv preprint https://doi.org/10.48550/arXiv.2401.05080 (2024)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Cihang Huang[1] · Zhucui Jing[2]

✉ Cihang Huang
hch20061006@163.com

[1] Weihai International College, Beijing Jiaotong University, Weihai 264401, Shandong, China

[2] Business Administration, Beijing Jiaotong University, Beijing 100044 , China